

TRIPWIRE — Execution Doc

Team Saturn · First Commit Hackathon

(Architecture diagram: see [tripwire_architecture.png](#), shared separately)

1. Problem Statement

India loses more money to electricity theft than any other country in the world — over ₹1.32 lakh crore every year. The reason isn't that smaller power companies don't try. It's that they don't see enough theft cases on their own to learn what it looks like, and they're legally not allowed to share customer data with other utilities to fill that gap. So every small utility solves the same problem alone, from scratch, while a handful of well-resourced ones pull ahead. We built a system where power companies can share what a theft pattern *looks like* — never the customer data itself — so a small utility can catch theft as well as a big one, without anyone's privacy being touched.

2. What the Demo Shows

Three simulated substations, running on real theft-labelled consumer data. One of them has very little data and starts out catching theft badly — we show its detection number live. It generates a theft signature, and that signature has to pass a policy check before it can be shared: if it still carries a customer ID, it gets **blocked on screen**, gets rewritten, and the clean version passes through. The moment the weak site checks new anomalies against what the other two sites have already shared, its detection number **visibly improves** — before and after, same site, on screen. That number moving is the proof.

3. Architecture

See diagram above. In four lines:

Each site is a **Strands agent** running its own detector, wrapped around the same private data it always had. When it flags something, it strips out every identifying field and turns it into an abstract signature. That signature hits a **Cedar policy gate** — permit or deny, logged either way, no exceptions. Permitted signatures land in a shared **OpenSearch** store that every site can check against without retraining anything — the pooled knowledge grows, the models don't move.

Research grounding: detection method built on the SGCC dataset (Zheng et al., IEEE) and the EnsembleNTLDetect framework (Kulkarni et al., ICDMW 2021); the shared store is framed on Google Research's Titans — test-time memory, no retraining required.

3a. The Drift / Swarm Mechanism

This is the part that makes it more than "detect, then upload everything."

Drift = the trigger. Each site's detector isn't just scoring every reading — it's watching for *surprise*: consumption that deviates sharply from what that site normally expects. A small deviation is noise and gets ignored. A sharp one — a load-shape that doesn't match anything the site has seen — is what actually fires. That threshold is what decides whether something becomes a candidate signature at all. Without it, every site floods the shared store with low-value noise and the pooled memory becomes useless.

Swarm = no central brain. There is no master model deciding what matters across all sites. Each site is an independent Strands agent making its own call about its own data. When one site's drift trigger fires, it broadcasts outward to its peers — not because a coordinator told it to, but because that's what the agent does when it detects something surprising. Peers receive, check, and adapt independently, the same way the sharing site did. The intelligence is distributed across the sites, not centralised in one place — that's what makes it a swarm rather than a client-server system.

Why this matters for the low-data site specifically: a site with too few theft cases to trust its own threshold benefits the moment a *peer's* drift trigger fires, because it's now checking against a pattern it never had enough data to learn on its own. That's the entire mechanism behind the before/after number in the demo — it isn't retraining, it's a low-data site catching a peer's surprise signal.

4. Tasks — Heavy First

Person 1 — Detection

- Split SGCC dataset into 3–4 synthetic sites, uneven data volume
- Build the per-site anomaly detector (isolation forest / lightweight classifier on load-shape features)
- Run the **isolated baseline** — the weak number, before sharing
- Run the **isolated-vs-pooled comparison** once sharing works — this is the one number judges score
- Own this number end to end: if it's flat Friday, this is the person who adjusts feature design, not the whole team

Person 2 — Policy & Abstraction

- Build the abstraction layer: anomaly → signature vector, strip every identifying field
- Write Cedar policies: deny raw records/IDs, permit signatures under a granularity threshold
- Build and prove the **block-then-pass** case — a signature that leaks an ID gets denied, a sanitised version gets permitted
- Log every policy decision, make it inspectable

Person 3 — Orchestration & Memory

- Wire each site as a Strands agent: detect → abstract → request-to-share → receive → re-check
- Stand up OpenSearch as the shared pattern store
- Connect Person 1's detector and Person 2's policy gate into one running loop, not three separate scripts
- Make the pooled-memory check actually change Person 1's detection number

These three tasks run in parallel from Thursday and meet in the middle by Friday evening. This is 90%+ of the weekend's work.

5. UI/UX — Shared, Last, Minimal

Not a person's job. Not a build track. A joint 1–2 hour pass on Sunday morning, after the pipeline is frozen:

- Terminal/console output showing the detect → block → pass → improve sequence in real time, or a simple before/after plot
- No dashboard, no styled page, no separate frontend codebase
- Whoever finishes their core task first on Saturday helps here — it is not reserved for one person's whole weekend

The video is built from this real output, not from a designed interface.

6. Deliverables

- **Public GitHub repo**, first commit dated 17 Sept, all three with push access
- **Working pipeline**: detect → abstract → policy gate → shared store → improved detection, runnable end to end
- **One measured result**: isolated vs. pooled detection rate on the low-data site
- **Block-then-pass proof**, logged and reproducible

- **Three-minute demo video** — real console output, not a UI
- **README / write-up**: problem, architecture, the number, what we learned, honest caveats (synthetic site split, not live DISCOM data)
- **Submitted Sunday afternoon**, not at the deadline